

由上可知，我们的后置运算符调用各自的前置版本来完成实际的工作。例如后置递增运算符执行

```
++*this
```

该表达式调用前置递增运算符，前置递增运算符首先检查递增操作是否安全，根据检查的结果抛出一个异常或者执行递增 `curr` 的操作。假定通过了检查，则后置函数返回事先存好的 `ret` 的副本。因此最终的效果是，对象本身向前移动了一个元素，而返回的结果仍然反映对象在未递增之前原始的值。



因为我们不会用到 `int` 形参，所以无须为其命名。

显式地调用后置运算符

如在第 491 页介绍的，可以显式地调用一个重载的运算符，其效果与在表达式中以运算符的形式使用它完全一样。如果我们想通过函数调用的方式调用后置版本，则必须为它的整型参数传递一个值：

```
StrBlobPtr p(a1);           // p 指向 a1 中的 vector
p.operator++(0);             // 调用后置版本的 operator++
p.operator++();               // 调用前置版本的 operator++
```

尽管传入的值通常会被运算符函数忽略，但却必不可少，因为编译器只有通过它才能知道应该使用后置版本。

569

14.6 节练习

练习 14.27: 为你的 `StrBlobPtr` 类添加递增和递减运算符。

练习 14.28: 为你的 `StrBlobPtr` 类添加加法和减法运算符，使其可以实现指针的算术运算（参见 3.5.3 节，第 106 页）。

练习 14.29: 为什么不定义 `const` 版本的递增和递减运算符？

14.7 成员访问运算符

在迭代器类及智能指针类（参见 12.1 节，第 400 页）中常常用到解引用运算符（`*`）和箭头运算符（`->`）。我们以如下形式向 `StrBlobPtr` 类添加这两种运算符：

```
class StrBlobPtr {
public:
    std::string& operator*() const
    { auto p = check(curr, "dereference past end");
      return (*p)[curr];           // (*p) 是对象所指的 vector
    }

    std::string* operator->() const
    { // 将实际工作委托给解引用运算符
      return & this->operator*();
    }

    // 其他成员与之前的版本一致
}
```

解引用运算符首先检查 `curr` 是否仍在作用范围内, 如果是, 则返回 `curr` 所指元素的一个引用。箭头运算符不执行任何自己的操作, 而是调用解引用运算符并返回解引用结果元素的地址。



箭头运算符必须是类的成员。解引用运算符通常也是类的成员, 尽管并非必须如此。

值得注意的是, 我们将这两个运算符定义成了 `const` 成员, 这是因为与递增和递减运算符不一样, 获取一个元素并不会改变 `StrBlobPtr` 对象的状态。同时, 它们的返回值分别是非常量 `string` 的引用或指针, 因为一个 `StrBlobPtr` 只能绑定到非常量的 `StrBlob` 对象 (参见 12.1.6 节, 第 421 页)。

这两个运算符的用法与指针或者 `vector` 迭代器的对应操作完全一致:

```
StrBlob a1 = {"hi", "bye", "now"};
StrBlobPtr p(a1);                // p 指向 a1 中的 vector
*p = "okay";                      // 给 a1 的首元素赋值
cout << p->size() << endl;        // 打印 4, 这是 a1 首元素的大小
cout << (*p).size() << endl;      // 等价于 p->size()
```

对箭头运算符返回值的限定

570

和大多数其他运算符一样 (尽管这么做不太好), 我们能令 `operator*` 完成任何我们指定的操作。换句话说, 我们可以让 `operator*` 返回一个固定值 42, 或者打印对象的内容, 或者其他。箭头运算符则不是这样, 它永远不能丢掉成员访问这个最基本的含义。当我们重载箭头时, 可以改变的是箭头从哪个对象当中获取成员, 而箭头获取成员这一事实则永远不变。

对于形如 `point->mem` 的表达式来说, `point` 必须是指向类对象的指针或者是一个重载了 `operator->` 的类的对象。根据 `point` 类型的不同, `point->mem` 分别等价于

```
(*point).mem;                    // point 是一个内置的指针类型
point.operator()->mem;            // point 是类的一个对象
```

除此之外, 代码都将发生错误。 `point->mem` 的执行过程如下所示:

1. 如果 `point` 是指针, 则我们应用内置的箭头运算符, 表达式等价于 `(*point).mem`。首先解引用该指针, 然后从所得的对象中获取指定的成员。如果 `point` 所指的类型没有名为 `mem` 的成员, 程序会发生错误。
2. 如果 `point` 是定义了 `operator->` 的类的一个对象, 则我们使用 `point.operator->()` 的结果来获取 `mem`。其中, 如果该结果是一个指针, 则执行第 1 步; 如果该结果本身含有重载的 `operator->()`, 则重复调用当前步骤。最终, 当这一过程结束时程序或者返回了所需的内容, 或者返回一些表示程序错误的信息。



重载的箭头运算符必须返回类的指针或者自定义了箭头运算符的某个类的对象。

14.7 节练习

练习 14.30: 为你的 `StrBlobPtr` 类和在 12.1.6 节练习 12.22 (第 423 页) 中定义的 `ConstStrBlobPtr` 类分别添加解引用运算符和箭头运算符。注意: 因为 `ConstStrBlobPtr` 的数据成员指向 `const vector`, 所以 `ConstStrBlobPtr` 中的运算符必须返回常量引用。

练习 14.31: 我们的 `StrBlobPtr` 类没有定义拷贝构造函数、赋值运算符及析构函数, 为什么?

练习 14.32: 定义一个类令其含有指向 `StrBlobPtr` 对象的指针, 为这个类定义重载的箭头运算符。



14.8 函数调用运算符

571

如果类重载了函数调用运算符, 则我们可以像使用函数一样使用该类的对象。因为这样的类同时也能存储状态, 所以与普通函数相比它们更加灵活。

举个简单的例子, 下面这个名为 `absInt` 的 `struct` 含有一个调用运算符, 该运算符负责返回其参数的绝对值:

```
struct absInt {
    int operator()(int val) const {
        return val < 0 ? -val : val;
    }
};
```

这个类只定义了一种操作: 函数调用运算符, 它负责接受一个 `int` 类型的实参, 然后返回该实参的绝对值。

我们使用调用运算符的方式是令一个 `absInt` 对象作用于一个实参列表, 这一过程看起来非常像调用函数的过程:

```
int i = -42;
absInt absObj;                // 含有函数调用运算符的对象
int ui = absObj(i);            // 将 i 传递给 absObj.operator()
```

即使 `absObj` 只是一个对象而非函数, 我们也能“调用”该对象。调用对象实际上是在运行重载的调用运算符。在此例中, 该运算符接受一个 `int` 值并返回其绝对值。



函数调用运算符必须是成员函数。一个类可以定义多个不同版本的调用运算符, 相互之间应该在参数数量或类型上有所区别。

如果类定义了调用运算符, 则该类的对象称作**函数对象** (function object)。因为可以调用这种对象, 所以我们说这些对象的“行为像函数一样”。

含有状态的函数对象类

和其他类一样, 函数对象类除了 `operator()` 之外也可以包含其他成员。函数对象类通常含有一些数据成员, 这些成员被用于定制调用运算符中的操作。

举个例子, 我们将定义一个打印 `string` 实参内容的类。默认情况下, 我们的类会将